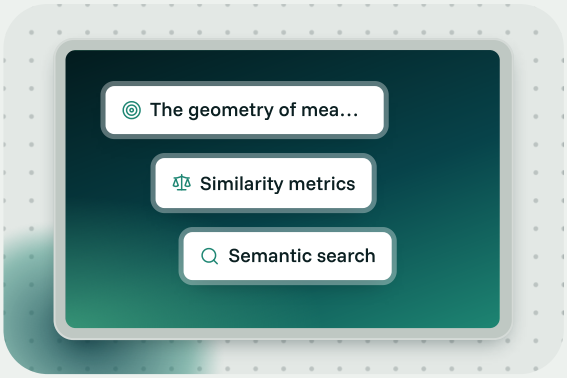
Last updated August 2026

Embeddings

How text becomes numbers, why similarity in vector space mirrors similarity in meaning, and the infrastructure that makes retrieval possible.



13 min read · Beginner-friendly with advanced sections

Pascal Academy
AI fundamentals · Part 9 of 10

What Is an Embedding?

Language models cannot read text and they largely process numbers. Before any model can generate, classify, or retrieve text, the text must be converted into a numerical representation that the model can work with. An embedding is that representation.

An embedding is a list of numbers, called a vector, that represents the meaning of a piece of text. The key property of an embedding is that text with similar meaning produces vectors that are close together in the vector space. Two sentences about the same topic will have vectors that are near each other, even if they use entirely different words. Two sentences about unrelated topics will have vectors that are far apart.

This is a different kind of representation from token IDs, which we covered in the first guide. A token ID is just an arbitrary number assigned to a token. The ID for the word cat might be 8420, and the ID for dog might be 3117. These numbers carry no information about the relationship between cats and dogs. Embeddings are different. The embedding for cat and the embedding for dog are close together in vector space because they are semantically related. The embedding for cat and the embedding for refrigerator are far apart because they are unrelated.

Key principle

Embeddings convert meaning into geometry. The distance between two vectors reflects the semantic distance between the texts they represent. This is what makes embeddings useful for search, clustering, classification, and retrieval. You can find relevant content by measuring distance in vector space rather than matching keywords.

How Embeddings Work

An embedding model takes text as input and produces a fixed-length vector as output. The vector might have 384, 768, 1536, or 3072 dimensions, depending on the model. Each dimension is a single number, and the full vector is a point in a high-dimensional space.

The embedding model is trained on large amounts of text using a simple objective. Texts that appear in similar contexts should get similar vectors, and texts that appear in different contexts should get different vectors. The model learns to produce vectors that capture semantic relationships by seeing millions of examples of which texts co-occur and which do not.

Go deeper

How embedding models are trained

Optional

Embedding models use a contrastive learning objective. During training, the model sees pairs or groups of texts. Some pairs are positive, meaning they are semantically related, like a question and its answer, or two sentences from the same paragraph. Other pairs are negative, meaning they are unrelated. The model is trained to produce vectors where positive pairs are close together and negative pairs are far apart. This is repeated across millions of examples, and the model gradually learns a vector space that captures semantic relationships. Some modern embedding models, like those based on the contrastive language-image pretraining approach, extend this to multiple modalities, producing shared vector spaces where text and images can be compared directly.

Metric	How It Works
Cosine similarity	Measures the angle between two vectors, ignoring their magnitude. Ranges from -1 to 1, where 1 means identical direction. The common choice for text embeddings because the length of the vector matters less than its direction.
Dot product	Multiplies corresponding elements of two vectors and sums the results. Faster to compute than cosine similarity but sensitive to vector magnitude. Works well when vectors are normalised to unit length, in which case dot product equals cosine similarity.
Euclidean distance	Measures the straight-line distance between two points in vector space. Less commonly used for text embeddings because it is sensitive to vector magnitude and does not align as well with semantic similarity.

The Geometry of Meaning

The remarkable thing about embeddings is that the vector space they create has meaningful structure. Directions in the space correspond to semantic concepts. For example, in a well-trained embedding model, the direction from the vector for man to the vector for woman roughly corresponds to gender. Applying that same direction to the vector for king gives a vector close to queen. This is the classic king minus man plus woman equals queen example, and it demonstrates that embeddings capture relationships between words, not just individual word meanings.

This geometric structure extends to phrases and sentences, not just individual words. The embedding of a question and the embedding of the answer to that question tend to be close in vector space, because they discuss the same topic from different angles. This is what makes embeddings work for retrieval. You embed the user's question and find documents whose embeddings are nearby.

Similarity Metrics

To find the closest vectors to a query vector, you need a way to measure distance or similarity between vectors. Three metrics are commonly used.

In practice, cosine similarity is the standard choice for text embeddings. It is what most vector databases use as their default metric, and it is what embedding models are typically optimised for during training.

What Embeddings Are Used For

Embeddings are a foundational building block for many AI applications. Any task that involves finding similar text, grouping related content, or representing text as numbers for a machine learning model relies on embeddings.



Pascal Academy

Pascal Academy's embeddings and vector search course covers embedding model selection, vector database configuration, and production deployment patterns. Reach out to learn about cohort programmes for your engineering team.

Ask about team training

Semantic Search and RAG

As covered in the RAG guide, embeddings power the retrieval step of a RAG system. Documents are chunked, each chunk is embedded, and the vectors are stored in a vector database. When a user asks a question, the question is embedded using the same model, and the system finds chunks whose vectors are closest to the question vector. This is semantic search. It finds relevant content based on meaning rather than keyword matching, so a question about refund policies can retrieve a document titled return and exchange guidelines even though no words overlap.

Clustering and Classification

Embeddings let you group similar documents together without labelled training data. A clustering algorithm like k-means applied to document embeddings will group documents by topic, because documents on the same topic have nearby vectors. Similarly, a classification model can use embeddings as input features. Instead of training a model on raw text, you embed the text and train a classifier on the vectors. This is faster and often more accurate than training from scratch, because the embedding model has already learned useful representations of language.

Recommendation

If you embed product descriptions, user reviews, and search queries into the same vector space, you can recommend products by finding items whose embeddings are close to what the user has shown interest in. The same principle applies to content recommendation, where articles or videos are embedded and compared to the user's reading or viewing history.

Deduplication and Entity Resolution

When integrating data from multiple sources, the same entity often appears with slightly different text. Two product listings might describe the same item with different wording. Embedding both descriptions and measuring their similarity lets you identify near-duplicates automatically. This is useful for data cleaning, knowledge graph construction, and entity resolution.

Embedding Models

Several embedding models are widely used, each with different characteristics in terms of dimensionality, performance, and cost.

Model	Dimensions	Notes
OpenAI text-embedding-3-large	3072	High performance, supports dimensionality reduction for faster search at a small accuracy cost. Proprietary, API-based.
OpenAI text-embedding-3-small	1536	Lower cost than the large variant with good performance for most use cases. Proprietary, API-based.
Cohere embed-v4	1536	Strong multilingual performance. Supports both text and image embeddings in a shared space. Proprietary, API-based.
Google Gemini embedding-001	3072	Multilingual, supports up to 2048 input tokens. Integrated with Google Cloud. Proprietary, API-based.
BGE (BAAI General Embedding)	768 or 1024	Open source, available in multiple sizes. Strong performance on benchmarks. Can be self-hosted for data privacy.
E5 (Microsoft)	768 or 1024	Open source, trained with contrastive learning. Good multilingual support. Can be self-hosted.
MiniLM (Sentence Transformers)	384	Small, fast, and lightweight. Lower accuracy than larger models but suitable for prototyping and low-latency applications. Open source.

The choice of embedding model affects retrieval quality, cost, and latency. Larger models with more dimensions generally produce better embeddings but are slower and more expensive. The right choice depends on the use case, the volume of data, and whether data privacy requirements favour self-hosted open source models over proprietary API-based ones.

Vector Databases

Embeddings are only useful if you can search through them efficiently. A vector database stores embedding vectors alongside their original text and metadata, and provides fast similarity search across millions or billions of vectors.

Database	Characteristics
Pinecone	Managed service, no infrastructure to operate. Supports filtering, hybrid search, and automatic indexing. Proprietary.
Weaviate	Open source with managed cloud option. Supports hybrid search combining vector and keyword retrieval. Includes built-in modules for common embedding models.
Qdrant	Open source, written in Rust for performance. Supports filtering, payload storage, and quantisation for memory efficiency.
Milvus	Open source, designed for scale to billions of vectors. Distributed architecture. Supports multiple index types.
Chroma	Open source, lightweight, designed for developer simplicity. Good for prototyping and smaller-scale applications.
pgvector	PostgreSQL extension for vector search. Lets you store vectors alongside relational data in the same database. Good for applications that already use PostgreSQL.

Brute Force vs Approximate Search

The simplest way to find the closest vectors to a query is to compute the similarity between the query and every vector in the database, then return the top results. This is called brute force search, and it is perfectly accurate. For a database of 10,000 vectors, it is fast. For a database of 100 million vectors, it is too slow for real-time queries.

Vector databases solve this with approximate nearest neighbour search. Instead of checking every vector, they use indexing structures that partition the vector space, allowing the search to skip large portions of the database that are unlikely to contain the closest vectors. The result is a small loss of accuracy, meaning the exact nearest neighbours might not always be found, in exchange for a large speed improvement.

Popular Vector Databases

Several vector databases are commonly used, each with different strengths in terms of scale, features, and ease of deployment.



Go deeper

Indexing algorithms

Optional

Several indexing algorithms power approximate nearest neighbour search. HNSW, or hierarchical navigable small world, builds a graph where each vector connects to its nearby neighbours. Search traverses the graph from a starting point, moving toward vectors that are closer to the query. IVF, or inverted file index, clusters vectors into groups and only searches the clusters closest to the query. PQ, or product quantisation, compresses vectors into smaller representations to reduce memory usage. Many vector databases combine these techniques, using IVF to narrow the search space and HNSW for fast traversal within each cluster. The choice of index affects the trade-off between speed, accuracy, and memory usage.

Embedding Quality and Evaluation

Not all embedding models are equally good. Choosing the right model requires understanding what makes an embedding good and how to measure it.

What Makes a Good Embedding

A good embedding captures semantic similarity accurately. Sentences that mean the same thing should have vectors that are close together. Sentences that mean different things should have vectors that are far apart. The embedding should also generalise across topics, languages, and writing styles. An embedding model trained only on news articles may perform poorly on legal text or code.

Evaluation Benchmarks

Embedding models are evaluated on standard benchmarks that test their ability to capture semantic relationships. The MTEB, or Massive Text Embedding Benchmark, evaluates models across several tasks including retrieval, classification, clustering, and semantic textual similarity. MS MARCO evaluates retrieval quality on real web search queries. BEIR provides a collection of datasets for evaluating zero-shot retrieval across domains.

Benchmark scores give a rough sense of model quality, but they do not always reflect performance on your specific data. The best approach is to build a small evaluation set of queries and relevant documents from your domain, embed them with candidate models, and measure which model retrieves the correct documents with the highest accuracy.

Practical Considerations



Pascal Academy

Pascal Academy's RAG engineering course covers embedding model selection, evaluation methodologies, and vector database deployment. Reach out to learn about team training options.

Ask about team training

Input Length

Embedding models have a maximum input length, typically measured in tokens. A model might accept up to 512 tokens, which is roughly 400 words. If you try to embed a longer document, the model will truncate it, potentially losing important information. This is why RAG systems chunk documents before

embedding, as covered in the RAG guide. The chunk size should fit within the embedding model's input limit.

Normalisation

Some embedding models produce normalised vectors, meaning each vector has a length of 1. Others produce unnormalised vectors. If you are using cosine similarity, normalisation does not matter because cosine similarity ignores magnitude. If you are using dot product, normalisation is important because dot product is affected by vector length. Check whether your model produces normalised vectors and configure your vector database accordingly.

Cost and Latency

Embedding text costs money and takes time. For API-based models, you pay per token for embedding. For a knowledge base of one million chunks averaging 200 tokens each, the embedding cost is 200 million tokens. This is a one-time cost unless the data changes. Query-time embedding is cheap because each query is a single short text. Latency at query time is dominated by the vector database search, not the embedding, unless the query is very long.

Multilingual Support

If your application handles multiple languages, choose an embedding model with multilingual support. These models are trained on text in many languages and produce a shared vector space where a question in Hindi and a document in English can be compared directly. Models without multilingual training will produce poor results when the query and the documents are in different languages.

Embeddings in the AI Stack

Embeddings sit at a specific layer in the AI application stack, and understanding where they fit helps clarify the relationship between the components covered in this series.

At the bottom is the language model itself, trained on trillions of tokens, which generates text. Above that is the embedding model, which converts text into vectors for search and retrieval. Above that is the vector database, which stores and searches those vectors. Above that is the RAG system, which orchestrates retrieval and generation. And above that are agents, which use all of these components to plan and execute multi-step workflows.

Embeddings are also used inside language models themselves. The Transformer architecture uses learned vector representations of tokens as its internal representation of text. The embedding models we have been discussing in this guide are specialised models trained specifically to produce good vectors for search and retrieval, but the underlying concept is the same. Converting text into vectors that capture meaning is fundamental to how all modern AI systems process language.

Limitations of Embeddings

Embeddings are powerful, but they have limitations that are worth understanding.

Summary

An embedding is a numerical vector that represents the meaning of a piece of text. Texts with similar meaning produce vectors that are close together in vector space, and texts with different meanings produce vectors that are far apart. This property lets you find relevant content by measuring vector distance rather than matching keywords. Embedding models are trained using contrastive learning, which teaches them to produce similar vectors for related texts and different vectors for unrelated texts. Cosine similarity is the standard metric for comparing vectors. Embeddings power semantic search and RAG, clustering and classification, recommendation, and deduplication. Vector databases store embeddings and provide fast approximate nearest neighbour search through indexing algorithms like HNSW and IVF. The choice of embedding model affects retrieval quality, cost, and latency, and should be evaluated on domain-specific data rather than relying on benchmark scores alone. Embeddings have limitations. They compress text into fixed-length vectors, losing detail. They struggle with negation and domain-specific vocabulary. They capture similarity but not relationship structure. For structured relationships, knowledge graphs are the appropriate tool.

Loss of Information

An embedding compresses a piece of text into a fixed-length vector. This compression loses information. A 500-word document and a 10-word sentence might both produce a 768-dimensional vector. The vector captures the dominant semantic content but discards details, nuance, and structure. This is why chunking matters. A chunk that is too large produces an embedding that averages over too many topics, diluting the signal.

No Understanding of Negation

Embeddings struggle with negation. The sentence I like this product and I do not like this product can produce similar embeddings because the words are nearly identical. The meaning is opposite, but the vector similarity is high. This is a known limitation of embedding-based retrieval and a reason why hybrid search, combining vector search with keyword search, can outperform pure vector search on certain queries.

Domain Sensitivity

Embedding models trained on general text may not capture domain-specific relationships well. Medical text, legal text, and code use specialised vocabulary and sentence structures that differ from general web text. A general-purpose embedding model may produce poor results on these domains. Fine-tuning an embedding model on domain-specific data, or choosing a model pretrained on relevant text, can improve performance.

No Relationship Structure

Embeddings capture similarity, not structure. They can tell you that two texts are related, but they cannot tell you how they are related. For structured relationships, as covered in the knowledge graphs guide, a graph representation is more appropriate. Embeddings and knowledge graphs are complementary. Embeddings handle semantic similarity, and knowledge graphs handle structured relationships.

About this guide

This guide is part of Pascal Academy's AI Fundamentals series, covering LLMs, prompt engineering, RAG, agents, context windows, knowledge graphs, hallucinations, reasoning models, embeddings, and MCP. The full series and hands-on courses are available at Pascal Academy. For teams looking to upskill, we offer custom cohort programmes tailored to your stack and use cases.

Frequently asked questions

What are embeddings in AI?

Embeddings are numerical representations of text where similar concepts are mapped to nearby points in a high-dimensional space. They allow a model to understand semantic similarity, so 'revenue growth' and 'top-line expansion' are recognized as related even though the words differ.

How are embeddings used in finance applications?

Embeddings power semantic search, document retrieval, and clustering. In finance, they enable searching across filings for relevant passages, finding similar companies based on business descriptions, and organizing research by topic. They are the backbone of retrieval systems that find information based on meaning rather than keywords.

How does Pascal use embeddings for financial research?

Pascal uses embeddings to index filings, transcripts, and your internal documents so that research agents can retrieve relevant information based on semantic meaning. When you ask about margin expansion, the system finds passages discussing profitability improvements even if those exact words are not used. This makes retrieval more accurate than keyword search alone.

What is the difference between embeddings and a vector database?

Embeddings are the numerical representations themselves. A vector database is the system that stores and searches through embeddings at scale. You generate embeddings from text, store them in a vector database, and query the database to find similar passages. They are two components of the same retrieval pipeline.